

Neural Network

MDE - Advanced Engineering Topics, ITESO

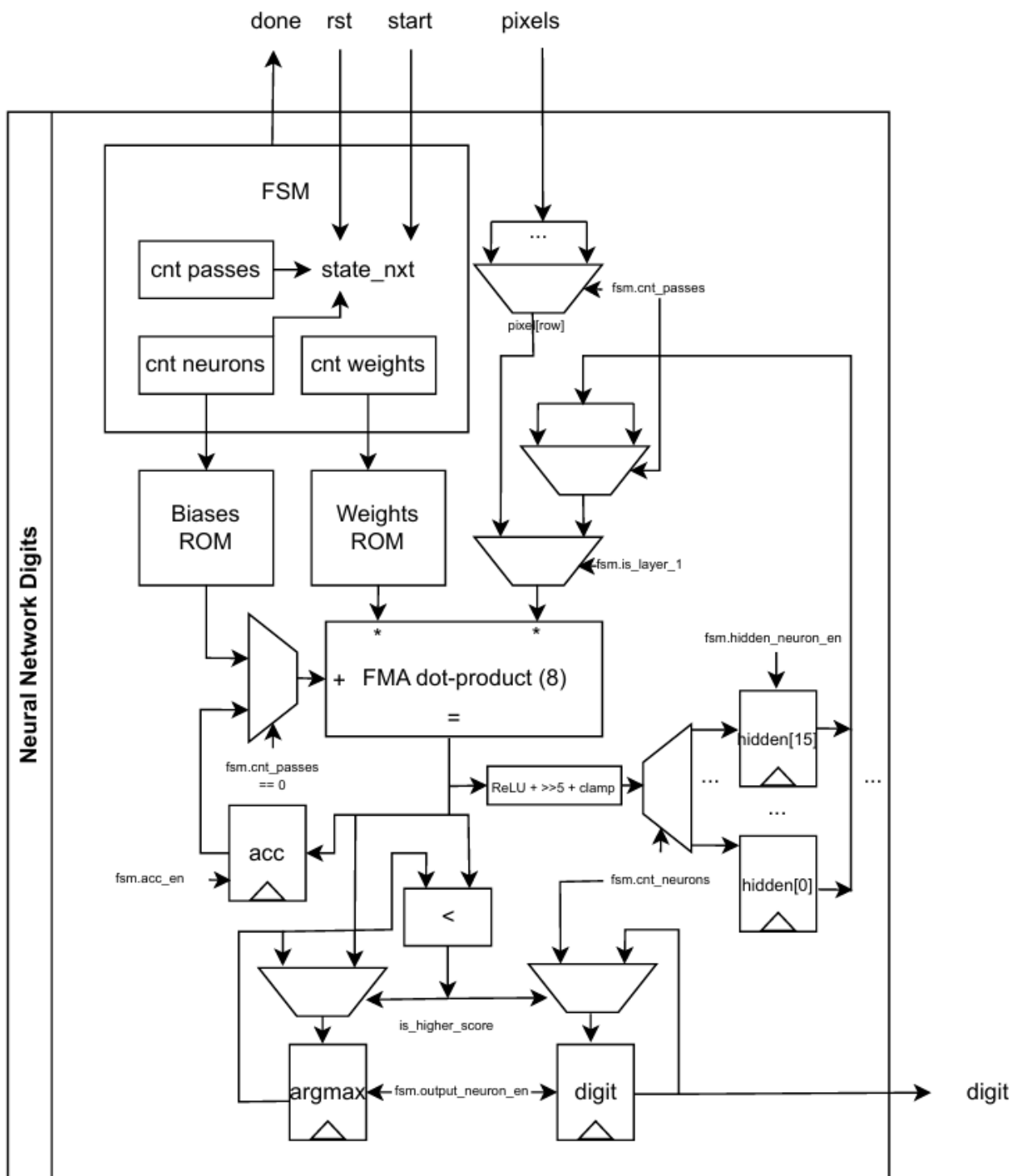
Gomez Enriquez, Gerardo

gerardo.gomez@iteso.mx

Guadalajara, Mexico

Implementation choices

Design space	Implementation choice	Trade-offs	Choice benefit	Choice downside
Datapath reutilization	Time-multiplexed	Area / Throughput	Less area	Less throughput
Internal accumulation style	Parallel multipliers with CSA tree (FMA DP)	Area / Latency / Timing / Complexity	Less latency, area & complexity than pipe. Better timing than chain	More area than chain
Multiplier vector width	N = 8	Area / Throughput / Latency	Balanced area for target performance	One image every 149 cycles
Combinational / Pipelined	Combinational	Area / Throughput / Latency	Less area & latency	Less throughput



Area / resource usage analysis

FMA dot-product

Operands width

Source A: 8-wide vector of 9-bit operands

- Multiplexed between: Pixel (4-bit) and hidden neuron activation (8-bit) + 1 bit to force unsigned / positive value

Source B: 8-wide vector of 8-bit weights (signed)

Source C: Single 32-bit operand

- Multiplexed between: Bias and accumulator (both widths 32-bit)

Internal structure

- 8 multipliers in parallel inside FMA dot-point (no FMA chain)
 - 1 Wallace tree per multiplier (each multiplier output is 2 operands)
 - 1 Wallace tree to reduce all multipliers “partial results”
 - 1 Carry Look-Ahead Adder as final CPA

Description	Instances	Operands in	Operands out	Levels
Per-multiplier Wallace tree	8 (one per multiplier)	6 each (5 Booth PPs + 1 two's-comp correction)	2 each	3 CSA
Final Wallace tree	1 (shared)	17 (8×2 mult outputs + 1 srcc)	2	6 CSA
Final CPA adder (CLA)	1 (shared)	2 (32-bit)	1 (32-bit result)	3 LCU

Area breakdown

FP DP Item	LUTs
Per multiplier (without Wallace tree)	~71 (70–73)
Wallace tree instance per multiplier	~69 (64–75)
Final Wallace tree	633
Final CPA(CLA)	134
Total	1,893

LUT usage comparison with different numbers of multipliers (N)

N	Operands into final tree ($2 \cdot N + 1$)	Final tree CSA levels ($\approx \log_{1.5}$)	Multipliers LUTs ($\approx 141 \times N$)	Final Wallace tree LUTs ($\approx 42.2 \times (\text{ops} - 2)$)	CLA LUTs (fixed)	Total LUTs ($\approx$)
1	3	1	141	42	134	~317
2	5	3	282	127	134	~543
4	9	4	564	295	134	~993
8	17	6	1,126	633	134	1,893
16	33	7	2,256	1,308	134	~3,698
64	129	11	9,024	5,359	134	~14,517

- These are approximations, not real synthesis for $N \neq 8$.
- Area scales near-linearly with N. However the final Wallace tree grows logarithmically, which is the non-linear part of the estimate.

FF usage

Signal	Bits	Comments
acc	32	
hidden_act	128	Hidden neuron layer activations (16×8)
argmax	32	
predicted_digit	4	
delayed_fma_dp_result	32	Registered fma_dp_result, decouples argmax compare from the FMA critical path
delayed_output_neuron_en	1	Registered output_neuron_en (paired with the delayed result)
delayed_sel_output_neuron	4	Registered sel_output_neuron (paired with the delayed result)
control_fsm.state	4	One-hot, re-encoded from 2-bit enum (4 states: IDLE, LAYER1, LAYER2, DONE)
control_fsm.counter_passes	4	Pass counter within a neuron (Max count = 8)
control_fsm.counter_neurons	5	Neuron counter (0–25 total neurons)
control_fsm.counter_weights	8	Weight-ROM address counter (0–147)
weights_rom	351	Implemented with LUTs (asynchronous read). Quartus inserted these registers during physical synthesis
biases_rom	0	Implemented with LUTs (asynchronous read).
RTL total	254	
Synthesis report total	619	
Gap	365	Number differs due to physical-synthesis optimizer: register retimer, duplication and removal

Resource usage summary

Component	LUTs	FFs
weights_rom	1,131	351
biases_rom	16	0
control_fsm	58	23
argmax_cmp	32	0
fma_dp	1,893	0
neural_network_digits (own logic)	420	245
Total	3,550	619

- The weights ROM dominates area (32% of the design) expected, since it's implemented as logic-based ROM due to asynchronous read requirements that dedicated memories in the FPGA does not support.
- The weights ROM output has very high fanout. Each of the 64 weight bits feeds into 8 parallel Booth-multiplier instances, each with multiple partial-product terms consuming that same bit. A wide, deep async-read mux tree driving that much fanout is the kind of shape that the physical-synthesis optimizer looks for register retiming and duplication for fanout as a legal way to shorten combinational delay or reduce net loading
- The net effect of the 351 registers Quartus added to weights_rom are not extra logical state. They're the tool exploiting otherwise-idle flip-flops already present in the LUT-based ROM's Logic Elements to help meet the timing target.

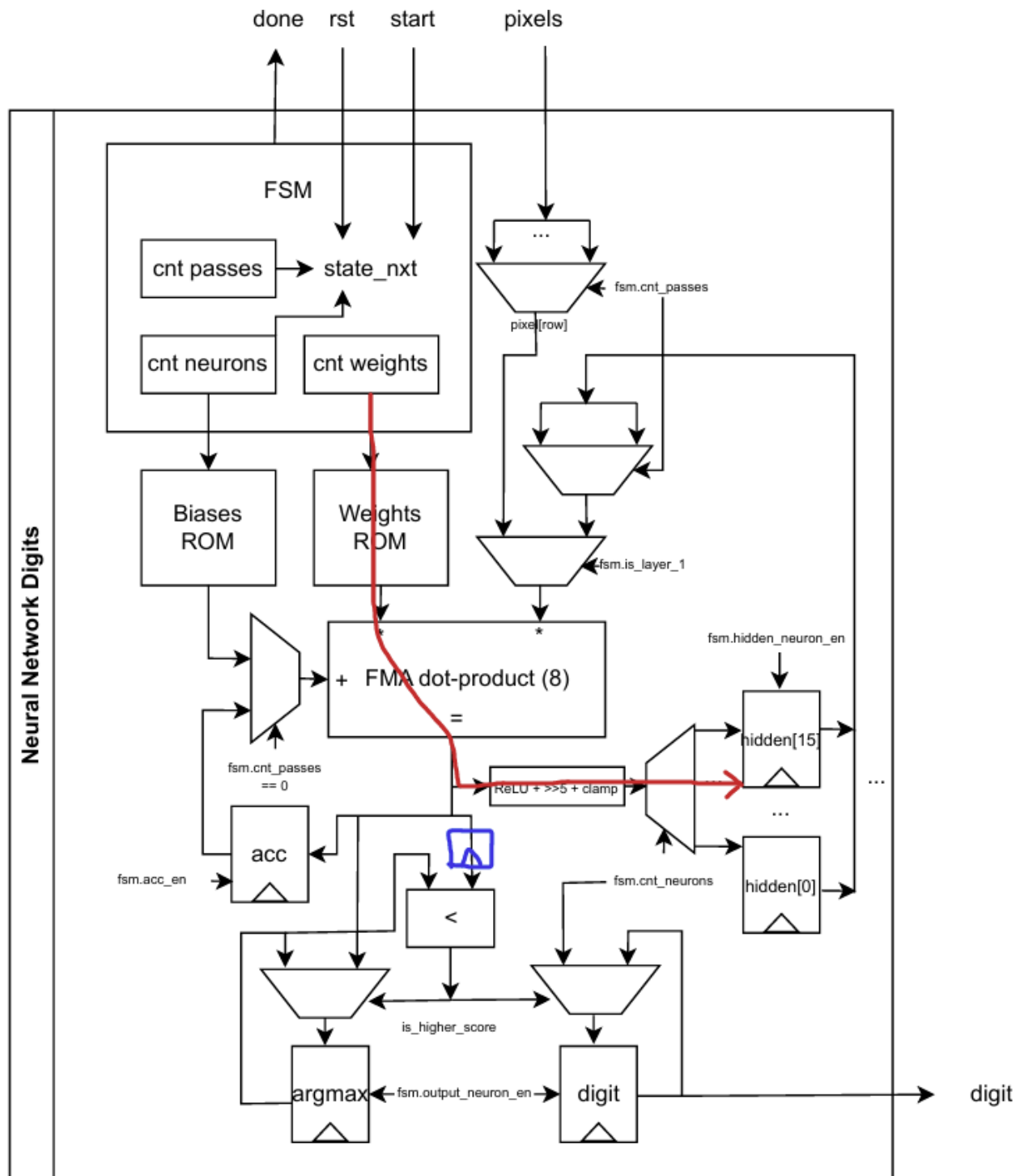
Timing analysis

Critical path

From *counter_weights[*]* → To *hidden_act[*][*]* — 26 logic levels, slack +0.419 ns

Description	Node	Count
Weight ROM address decode	weights_rom rom_data~696/697/698 combout	3
Booth PP-select mux (multiplier)	gen_mul[2]... Mux28~0/1 combout	2
Per-multiplier Wallace tree	gen_mul[2]... wallace_tree gen_lvl[0..2].gen_csa[0].csa .. cout/sum	3
Final Wallace tree	fma_dp wallace_tree gen_lvl[0..5].gen_csa[*].csa ... cout/sum	6
Carry look-ahead Adder	fma_dp cla gen_lvl[0..1].gen_lcu[*].lcu4 gg/c	7
Wide-OR reduction (clamp check in requantization)	WideOr0~1/2/3/6 combout	4
Hidden-activation writeback mux	hidden_act_nxt[0][3]~7 combout	1
Total		26

- The path traverses the entire dot-product engine end-to-end: ROM address decode → Booth PP mux → per-multiplier CSA reduction → shared final CSA reduction → CLA → requantization (WideOr, the clamp-to-255 check) → hidden-activation register.
- The earlier structural analysis that stacked 3 (per-mul CSA) + 6 (final CSA) + 3 (CLA LCU) levels dominates the critical path along with the asynchronous ROM read and requantization.



- In red, the critical path.
- In blue, the registered `fma_dp_result` (delayed_fma_dp_result), decouples `argmax` compare from the FMA critical path. Before delaying the signal that was the critical path. The delay was engineered to avoid affecting the final latency of the design.

Timing metrics summary

Metric	Value
Target frequency	50.0 MHz (20.000 ns period)
Fmax (Slow 1200 mV, 85 °C)	51.07 MHz
Worst setup slack	+0.419 ns (met)
Max LUT depth	28
Latency	149 cycles
Throughput	1 image every 149 cycles

- Timing closure was achieved at the 50 MHz target with positive slack (+0.419 ns), and the design's Fmax (51.07 MHz) sits just above that target.
- The optimization mode was set to high performance effort to achieve this frequency, otherwise the maximum frequency was 49 MHz.
- The retiming and register-duplication optimizations seen in weights_rom were the mechanisms Quartus used to reach this narrow positive slack at 50 MHz since the asynchronous ROM read sits in the critical path of the design.
- In an attempt to improve timing, the Wallace tree was optimized by removing the Wallace trees per multiplier and adding the partial products from all multipliers together in a single Wallace tree. The number of CSA levels decreased by one but somehow timing got worse and the change was reverted.

Conclusion

The chosen implementation is not well suited for high-performance systems: its combinational datapath limits the maximum frequency, and the resulting throughput is low. It does, however, offers a reasonable balance between area and throughput/latency for applications that do not demand high-rate image processing.